Project 2: Bayesian Classification and Regression for Data Analysis.

This continuous assessment is a project-style exercise on Bayesian Modeling, where you will experience how we leverage Bayesian modeling for both classification tasks and regression tasks. You will work on the cougar-face vs cougar-body classes for the classification task and the Computer Hardware (CPU Performance) dataset for the linear regression task.

You are required to implement the learning algorithms from scratch (except where explicitly stated), and provide a comparative analysis supported by quantitative and qualitative evidence.

Before you start the assignment, please read through the following guidelines so that you understand the requirements. Follow the guidelines strictly to avoid unnecessary mark deductions:

1. Implement your codes with Python (recommended) or MATLAB (or any other languages you deem comfortable with).
2. For questions that require you to show your code, make sure your codes are **clean** and **easily readable** (add meaningful comments, comments are **markable**). Embed your code into your answer with screenshots.
3. Your submission should be one single PDF file with the necessary code and results collated in a single file. Incorrect submission format will result in a **direct** 10 points (out of 100) deduction.
4. Do submit your project on Canvas before the deadline: **23:59 (SGT), 20 Mar 2026**. There will be **10** submissions attempts restricted for every student.
5. Policy on late submission: the deadline is a strict one, so please prepare and plan early and carefully. Any late submission will be deducted 10 points (out of 100) for every 24 hours.
6. This is an **individual** project, do **NOT** share your solutions with others, we have zero tolerance for **plagiarism**.
7. If you made use of any AI tools for your project (which I strongly recommend), please include an **AI Usage Statement** that states the AI tool used and how you have leveraged the AI tool.

Part 1: Bayesian Modeling for Image Classification with MLE and MAP (50%).

a. Data loading and feature construction (10%)

You will be provided with the "caltech_cougar.zip" dataset. For this project, convert all images to **grayscale** and resize them to a fixed resolution of 64×64 pixels. Each image must be flattened into a 1D feature vector $\mathbf{x} \in \mathbb{R}^D$, where $D = 4096$. In your report, provide basic dataset statistics, including sample counts per class for your chosen train/test split and a visualization of the "Mean Image" for each class. Your train versus test ratio should not exceed 75:25 (i.e., you cannot have more than 75% image for each class as your training image).

b. Bayesian Modeling with Varying Distribution Assumptions (MLE) (20%)

You will now implement two different models from scratch to compare how different distribution assumptions affect classification performance. **First**, implement a **Gaussian Naive Bayes** classifier by estimating the mean vector $\boldsymbol{\mu_C}$ and the variance vector $\boldsymbol{\sigma_C^2}$ for EACH class using the **MLE** formulas for a Gaussian distribution (show the derivation of the MLE formulas).

Second, re-evaluate the task by assuming each pixel intensity follows a **Laplace Distribution**, which is often more robust to outliers than the Gaussian distribution. You must estimate the location parameter $\boldsymbol{\mu}$ and the scale parameter $\boldsymbol{b}$ for each class using their respective MLE formulas:

$$\widehat{\boldsymbol{\mu}}_{\boldsymbol{Laplace}} = \text{median}(\boldsymbol{x_i}) \text{ and } \widehat{\boldsymbol{b}} = \frac{1}{N}\sum_{i=1}^{N}\left|\boldsymbol{x_i} - \widehat{\boldsymbol{\mu}}_{\boldsymbol{Laplace}}\right|$$

Evaluate both models on your test set, report the accuracy and confusion matrices, and discuss which distribution assumption (Gaussian vs. Laplace) provides better generalization for this specific image dataset.

c. Linear SVM evaluation and discussion (20%)

You will now incorporate **Prior Knowledge** into your Gaussian model using a unique **Prior Mean** $\boldsymbol{\mu_0}$ and **Prior Variance** $\boldsymbol{\sigma_0^2}$ vector based on your **Student ID**. Let S be the sequence of digits in your ID. Your prior mean $\boldsymbol{\mu_0}$ for all pixels should be the average value of S (scaled to the [0, 255] range), and your prior variance $\boldsymbol{\sigma_0^2}$ should be a vector where each element i is calculated as:

$$\boldsymbol{\sigma_{0,i}^2} = Var(S) \times \left(1 + \frac{i \pmod{\text{MaxDigit}}}{10}\right)$$

For example, if your ID digits average to 4.5, then $\mu_{0,i} = \frac{4.5}{9} \times 255 \approx 127.5$ for all i.

Implement the MAP estimation for the Gaussian:

$$\boldsymbol{\mu_{MAP}} = \frac{\boldsymbol{\sigma^2\mu_0} + n\boldsymbol{\sigma_0^2}\bar{\boldsymbol{x}}}{\boldsymbol{\sigma^2} + n\boldsymbol{\sigma_0^2}} \text{ and } \boldsymbol{\sigma_{MAP}^2} = \frac{\boldsymbol{\sigma^2\sigma_0^2}}{\boldsymbol{\sigma^2} + n\boldsymbol{\sigma_0^2}}$$

For simplicity, use the MLE variance obtained from Question b as the value for $\boldsymbol{\sigma^2}$. Compare the resulting test performance against your MLE-based Gaussian classifier. In your report, analyze the relationship between n (sample size) and the posterior variance $\boldsymbol{\sigma_{MAP}^2}$. **Explain** why the model's "certainty" increases as more data is observed and why MAP provides more **robust** parameter estimates than MLE when training samples are extremely limited.

Part 2: Bayesian Linear Regression & Uncertainty Estimation (50%).

a. Implementation of MLE and MAP Regression (25%)

You will use the "machine.data" (Computer Hardware) dataset (provided in the “computer+hardware.zip” zipfile to perform a regression task predicting the "Estimated Relative Performance" of various hardware configurations. **Firstly**, you are to implement the MLE for linear regression $Y = \theta X$ from scratch using the closed-form matrix solution:

$$\theta_{MLE} = (X^T X)^{-1} X^T Y.$$

Try to include a formal derivation demonstrating how this MLE solution is equivalent to the Ordinary Least Squares (**OLS**) criterion, specifically showing how the optimization of the Gaussian log-likelihood leads to the minimization of squared errors.

Secondly, you will develop a MAP estimation model by assuming a **Gaussian** prior on the weights $\theta \sim \mathcal{N}(0, \lambda^{-1} I)$. The MAP solution is given by:

$$\theta_{MAP} = (X^T X + \lambda I)^{-1} X^T Y.$$

To ensure unique results, your regularization parameter λ would be calculated by taking the sum of the digits in your Student ID and dividing it by 100 (for example, if your digits sum to 30, then $\lambda = 0.30$).

For **both** the MLE and MAP models, you would compute and report the Mean Squared Error (MSE) on the test set. You should explain how the addition of the λI term affects the stability of the matrix inversion, particularly when dealing with potentially ill-conditioned data.

b. Scikit-Learn Benchmarking and Comparative Analysis (25%)

Now, you would benchmark your "from-scratch" implementations against the standard Scikit-Learn library. You will train a “LinearRegression” model and a Ridge regression model (setting the α parameter to your unique λ value) with “sklearn.linear_model”. You would validate your work by comparing the final weight vectors θ and the resulting test MSE between your implementations and the library’s output. **Explain** the discrepancies between your model and the Scikit-Learn results in detail.

Further, you will perform a qualitative analysis of the model weights by plotting the magnitudes of the individual coefficients for both the MLE and MAP models. **Explain** why the MAP weights are generally smaller in magnitude than the MLE weights and how this "shrinkage" effect serves as a form of regularization. Further, analyze which model is more likely to achieve better generalization on unseen hardware data, grounding your conclusion in the **Bias-Variance Tradeoff** concepts and **explain** how the prior distribution helps mitigate the risk of overfitting.